

Instructions on how to install and use Git for the Lake Taney field camp

Git will be used during the Lake Taney field camp to work collaboratively on data processing and analysis. We recommend one of the two following options:

- Use Git in VS Code (recommended if you want to code in VS Code as well): follow Sect. 1
- Use GitHub Desktop (recommended if you want to use another Code Editor than VS Code, like Spyder for example): follow Sect. 2

If you are familiar with Git in the terminal (i.e., Git Bash), it is another option but less user-friendly.

Official documentation for more information:

- [Git Documentation](#)
- [VS Code Git Documentation](#)
- [GitHub Desktop Documentation](#)

Prerequisites:

- *You need to have a GitHub account to modify the GitHub repository. To create one, go to <https://github.com/signup>.*
- *In addition, at least one member of each group needs to be added as a collaborator to the repository. Share your GitHub username with the teachers.*

Table of Contents

1. Option 1 – Git in VS Code	3
1.1. Git Installation.....	3
1.2. VS Code Installation	3
1.3. Repository installation in VS Code	4
1.4. Usage	4
2. Option 2 – GitHub Desktop.....	6
2.1. Installation.....	6
2.2. Repository installation in GitHub Desktop	6
2.3. Usage	6
3. Workflow	9
4. Some useful git commands.....	11
4.1. Checking Status	11
4.2. Staging & Committing	11
4.3. Working with Remote Repos.....	12
4.4. Branching	12
4.5. Stashing (Temporary Save)	12
4.6. Undoing Changes	13
4.7. Inspecting & Debugging	13

1. Option 1 – Git in VS Code

1.1. Git Installation

If you do not have Git installed on your computer, you can:

- install GitHub Desktop by following the procedure in Sect. 2.1,
- or install Git by following Steps 1& 2 below.

Step 1 - Install Git

- **Windows:**

1. Download Git from [Git for Windows](#)
2. Run the installer.
3. Keep default settings unless you know you need changes.

- **macOS:**

Install with Homebrew, in a terminal: `brew install git`

Or download from: [Git Downloads](#)

- **Linux**

Ubuntu/Debian, in a terminal:

```
sudo apt update
```

```
sudo apt install git
```

Step 2 - Configure Git (First Time)

Run these commands in a terminal:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@example.com"
```

1.2. VS Code Installation

If you do not have VS Code installed on your computer:

1. Go to [Visual Studio Code Download](#)
2. Download the installer for your OS:
 - Windows
 - macOS
 - Linux
3. Run the installer and keep the default options.

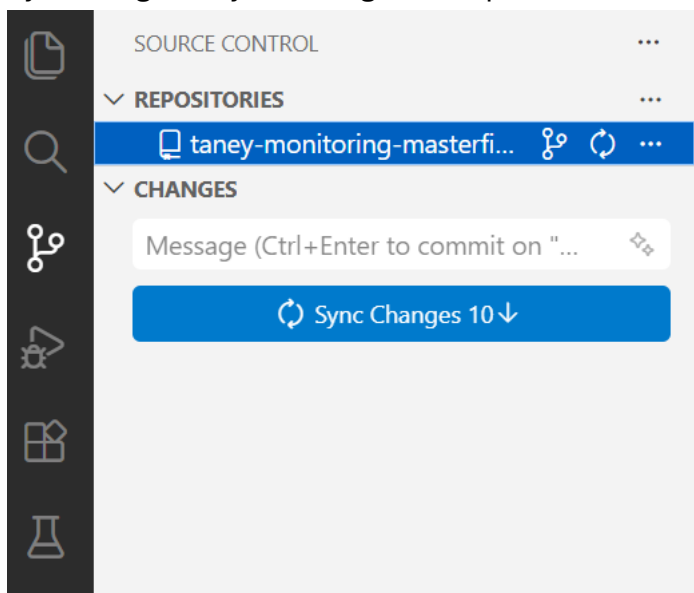
4. Launch VS Code.

1.3. Repository installation in VS Code

1. In VS Code, press:
 - Ctrl + Shift + P (Windows/Linux)
 - Cmd + Shift + P (macOS)
2. Type: `Git: Clone` and click it.
3. Paste the Repository URL: <https://github.com/tdoda/taney-monitoring-masterfieldcamp.git> and press Enter.
4. Select where you want the repository stored on your computer.
5. After cloning, VS Code asks: “Open the cloned repository?”. Click: “Open”.

1.4. Usage

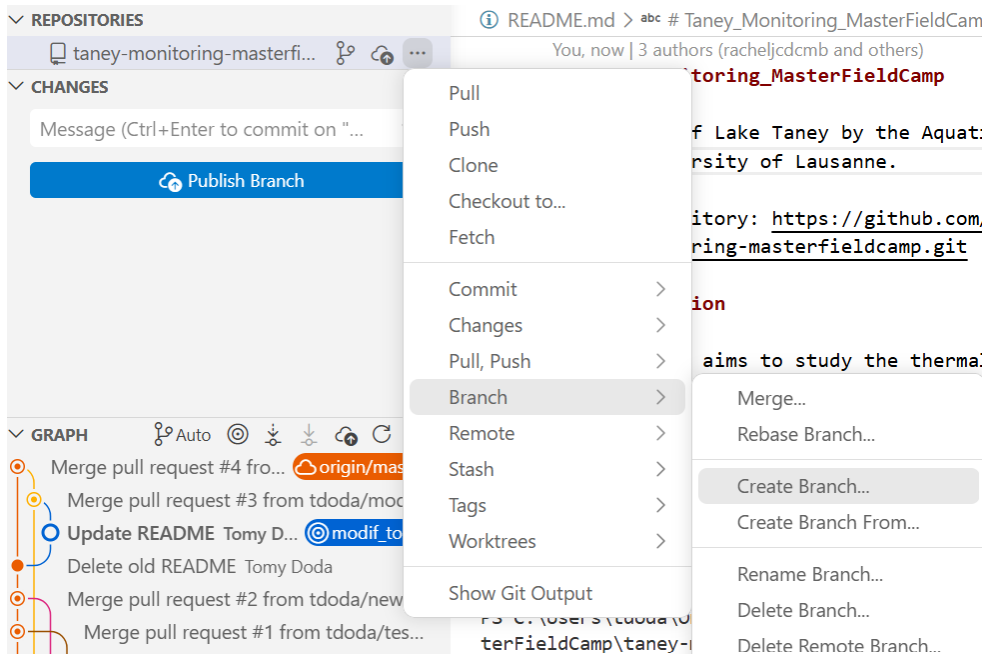
1. Make sure that the entire Lake Taney git repository is open in VS Code. If not, open it with “File” > “Open Folder”> select the folder corresponding to the repository.
2. To use Git in VS Code, click the **Source Control** icon on the left sidebar: The files that you changed are listed in the “CHANGES” section. Click on a file to see which changes have been made.
The previous commits are listed in the “GRAPH” section.
3. If modifications have been made on the remote repository, synchronize changes by clicking on “Sync Changes” and press “OK”.



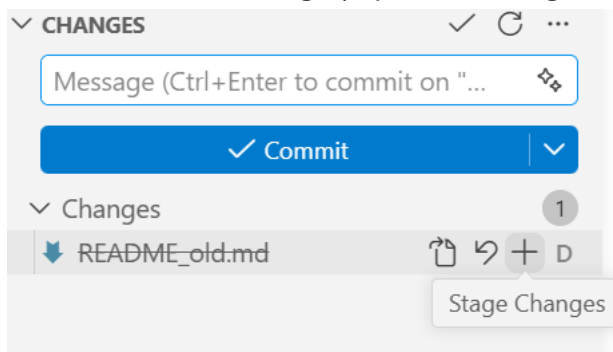
Once the local repository has been synchronized with the remote repository, your “master” branch (local) should match the “origin” branch (remote = cloud icon) in the “GRAPH” section:



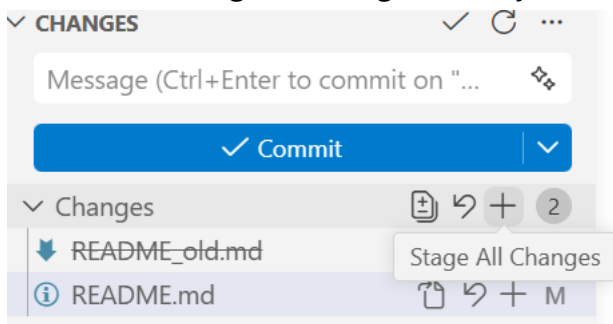
- Before making any change to the repository, create your own branch by clicking on “...” next to the repository name > “Branch” > “Create Branch”. Write a branch name and press Enter.



- Once you are happy with your changes, stage changes by clicking the + button next to the files to stage (equivalent to “git add” in the terminal).



You can also stage all changed files by clicking the + button next to “Changes”:



6. Commit the changes: enter a commit message at the top and click “Commit”.
7. Push to GitHub: click on “Publish Branch” if it is a new branch or “Sync Changes” otherwise.

2. Option 2 – GitHub Desktop

2.1. Installation

1. Go to: [GitHub Desktop Download](#)
2. Click **Download for Windows** or **Download for macOS**.

Windows Installation

1. Open the downloaded .exe file
2. The installer runs automatically
3. Wait for installation to finish
4. GitHub Desktop launches automatically

macOS Installation

1. Open the downloaded .zip
2. Drag **GitHub Desktop** into the **Applications** folder
3. Open the app

If macOS blocks it:

- Go to **System Settings → Privacy & Security**
- Click **Open Anyway**

Sign In to GitHub

1. Click **Sign in to GitHub.com**
2. Log in through your browser
3. Authorize GitHub Desktop

2.2. Repository installation in GitHub Desktop

1. Click “File” → “Clone repository”
2. Choose “URL” tab and paste the Repository URL:
<https://github.com/tdoda/taney-monitoring-masterfieldcamp.git>
3. Choose where the project will be saved on your computer.
4. Click “Clone”

2.3. Usage

1. Make sure the Lake Taney repository is selected in the top-left dropdown. If the repo is not there:
 - Click “File” → “Add local repository” (if already cloned), or

- Click “File” → “Clone repository” (Sect. 2.2)
2. View changes and history:
 - Go to the “Changes” tab (left panel)
 - Shows all modified, added, or deleted files
 - Click any file to view differences (diff view)
 - Go to the “History” tab
 - Shows all previous commits in a visual timeline
 3. If modifications have been made on the remote repository, synchronize changes:
 - Click “Fetch origin” (top bar) to check for remote updates
 - If changes exist, click “Pull origin” to update local repository

No local changes

There are no uncommitted changes in this repository. Here are some friendly suggestions for what to do next.



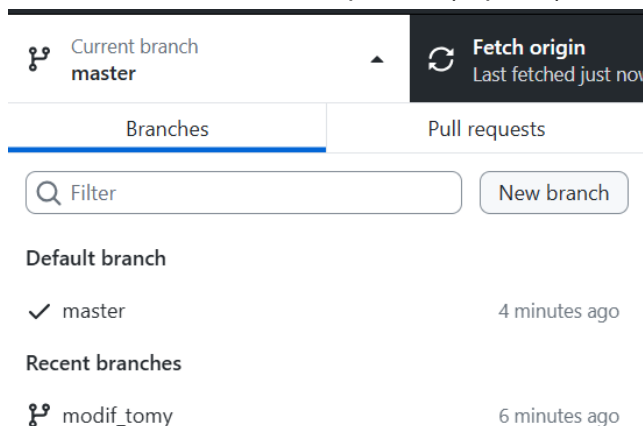
Pull 3 commits from the origin remote

The current branch (master) has commits on GitHub that do not exist on your machine.

Pull origin

Always available in the toolbar when there are remote changes or `Ctrl + Shift + P`

4. Before making any change to the repository, create your own branch by clicking the “Current Branch” dropdown (top bar) > “New Branch”.



Enter branch name and click “Create Branch”.

Create a branch

×

Name

modif_group1

Your new branch will be based on your currently checked out branch (master). master is the default branch for your repository.

Create branch

Cancel

- All modified files appear in Changes. Once you are happy with them, commit them by selecting the files you want in the commit and writing a commit message in the bottom-left panel > Click “Commit to <branch-name>”.

File Edit View Repository Branch Help

Current repository

taney-monitoring-masterfieldcamp

Current branch

modif_tomy

Changes 1

History

Filter

1 changed file

README.md

Update README

Description

+

New

Commit 1 file to modif_tomy

README.md

↑

21 21

22 22

23 23

24 24

25 25

26 26

24 24

25 25

26 26

27 27

28 28

29 29

↓

@@ -21,9 +21,9 @@ Co

cted in the field, c

Notes:

- The files corresponc

'yearmonthday' forme

- The 'EX0_...' probes

the RBR probes.

- The 'HOB0_P' file (p

rd.

+ The files corresponc

"yearmonthday" forme

+ The "EX0_..." probes

the RBR probes.

+ The "HOB0_P" file (p

rd.

Each type of data is

of pre-processing:

- Push to GitHub: click “Publish branch” if it is a new branch, or click “Push origin” otherwise.

8 | Page

No local changes

There are no uncommitted changes in this repository. Here are some friendly suggestions for what to do next.



Push commits to the origin remote

You have 1 local commit waiting to be pushed to GitHub.

Always available in the toolbar when there are local commits waiting to be pushed or

Ctrl + P

Push origin

3. Workflow

Each group works on different files if possible (to minimize conflicts) and one person per group is responsible of Git (committing and pushing changes to the group-specific branch). The person responsible needs to be added as a collaborator of the Github repository (share the GitHub username with the teachers).

The steps below should be followed:

1. Install the GitHub repository locally, as explained in Sect. 1.3 (VS Code) or in Sect. 2.2 (GitHub Desktop).
2. Each group synchronizes their local repository with the remote before making any modification.
3. Each group creates their own branch (e.g., “modif_group1”). See Sect. 1.4 (VS Code) or Sect. 2.3 (GitHub Desktop) for more details.
4. Add all past data to the “Data” folder of the repository: go to <https://drive.switch.ch/index.php/s/yyV253N9xIIceSs> , click on “download” and save to the “Data” folder. If new data has been collected, add the raw data and the metadata in “Data/data_type/date/Level0” folder (see the README file of the repository for more information).
5. Any change is staged and committed to the group-specific branch. The group-specific branch is pushed/published to the remote repository. See Sect. 1.4 (VS Code) or Sect. 2.3 (GitHub Desktop) for more details.
6. At the end of a working session when all changes should be combined, the group-specific branches are merged together using the “Compare & pull request” green button on GitHub.

 **modif_tomy** had recent pushes 3 seconds ago

[Compare & pull request](#)

 **master**  **2 Branches**  **0 Tags**



 **Add file**

 **Code**

- Compare the branches and resolve potential conflicts: select the branch of the other group you want to merge to as “base” and your own branch as “compare”.
- Add a title and comments to the pull request and click on “Create pull request”

 **base: master**  **compare: modif_tomy**  **Able to merge.** These branches can be automatically merged.



Add a title *

Update README



Add a description

Write Preview              

Project introduction has been updated in README file

 Markdown is supported  Paste, drop, or click to add files

[Create pull request](#)

- Click on “Merge pull request” > “Confirm merge” to merge the branches



tdoda commented [now](#)

Owner 

Project introduction has been updated in README file



 **Update README**

[ccc7f2d](#)



 **No conflicts with base branch**
Merging can be performed automatically.

[Merge pull request](#)

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

- Delete the branch that has been merged.



Pull request successfully merged and closed
You're all set — the `modif_tomy` branch can be safely deleted.

Delete branch

- Once all the group-specific branches are merged into a single branch, this branch is merged to the master branch using the “Pull requests” button on GitHub (same as step 6 with base branch = “master”).

4. Some useful git commands

The following commands can be used in the terminal of VS Code or in GitBash for example.

4.1. Checking Status

Command	What it does	When to use
git status	Shows changed files	Always before commits
git log	Shows commit history	Review history
git diff	Shows unstaged changes	Inspect edits
git diff --staged	Shows staged changes	Before commit
git show	Shows last commit details	Inspect commit

4.2. Staging & Committing

Command	What it does	When to use
git add <file>	Stages a file	Prepare commit
git add .	Stages all changes	Most common
git commit -m "msg"	Saves snapshot	After staging
git commit --amend	Fix last commit	Edit last message/files

4.3. Working with Remote Repos

Command	What it does	When to use
git remote -v	Shows remote URLs	Check repo connection
git fetch	Downloads updates (no merge)	Check remote changes safely
git pull	Fetch + merge/rebase	Get latest changes and override local changes
git pull --rebase	Re-applies your changes on top of remote	Cleaner history
git push	Upload commits	Send changes to GitHub

4.4. Branching

Command	What it does	When to use
git branch	Lists branches (add -r for remote branches)	View structure
git branch <name>	Creates branch	New feature
git checkout <branch>	Switch branch	Move between work
git merge <branch>	Merge branch into current	Combine work
git fetch --prune	Fetches updates and removes deleted remote branches	Keeps local branch list clean

4.5. Stashing (Temporary Save)

Command	What it does	When to use
git stash	Saves uncommitted changes	Before pull/switch
git stash pop	Restores stash	Resume work

Command	What it does	When to use
git stash list	Shows stash history	Manage saved work

4.6. Undoing Changes

Command	What it does	When to use
git restore <file>	Undo file changes	Before staging
git restore --staged <file>	Unstage file	Fix staging mistake

4.7. Inspecting & Debugging

Command	What it does	When to use
git show	Shows last commit details	Inspect commit
git blame <file>	Shows who changed lines	Debug code history
